

Payment Gateway — Complete Project Roadmap

Day 1 to final product. One document. Keep this open the entire build.

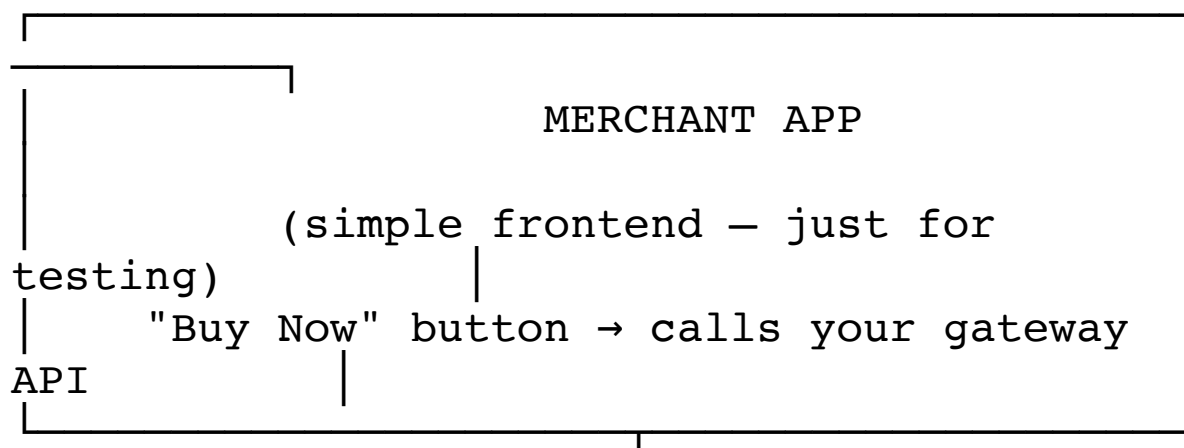
What You Are Building

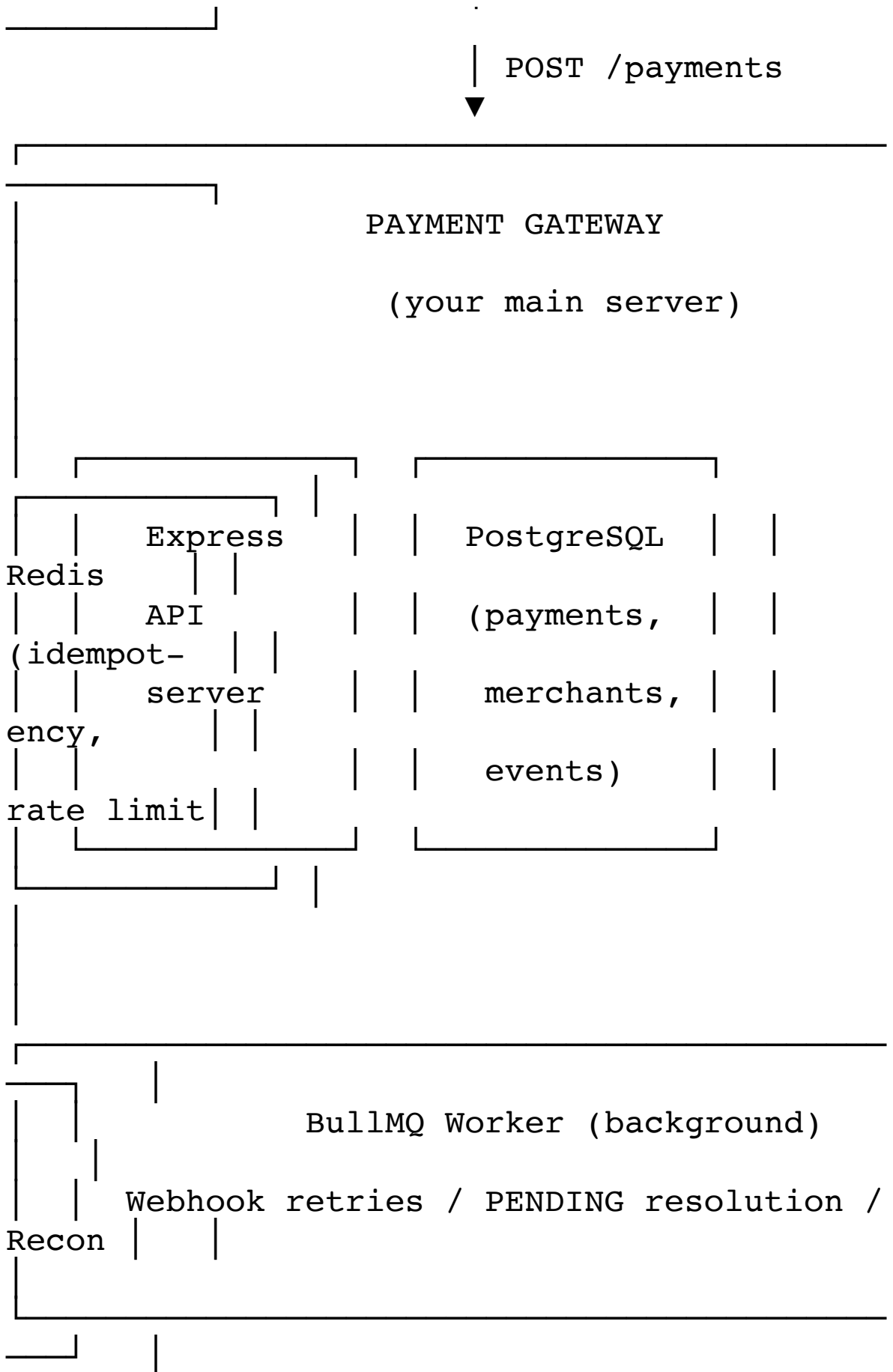
A payment gateway — the infrastructure layer that sits between a merchant (someone selling something) and a bank. Your gateway handles:

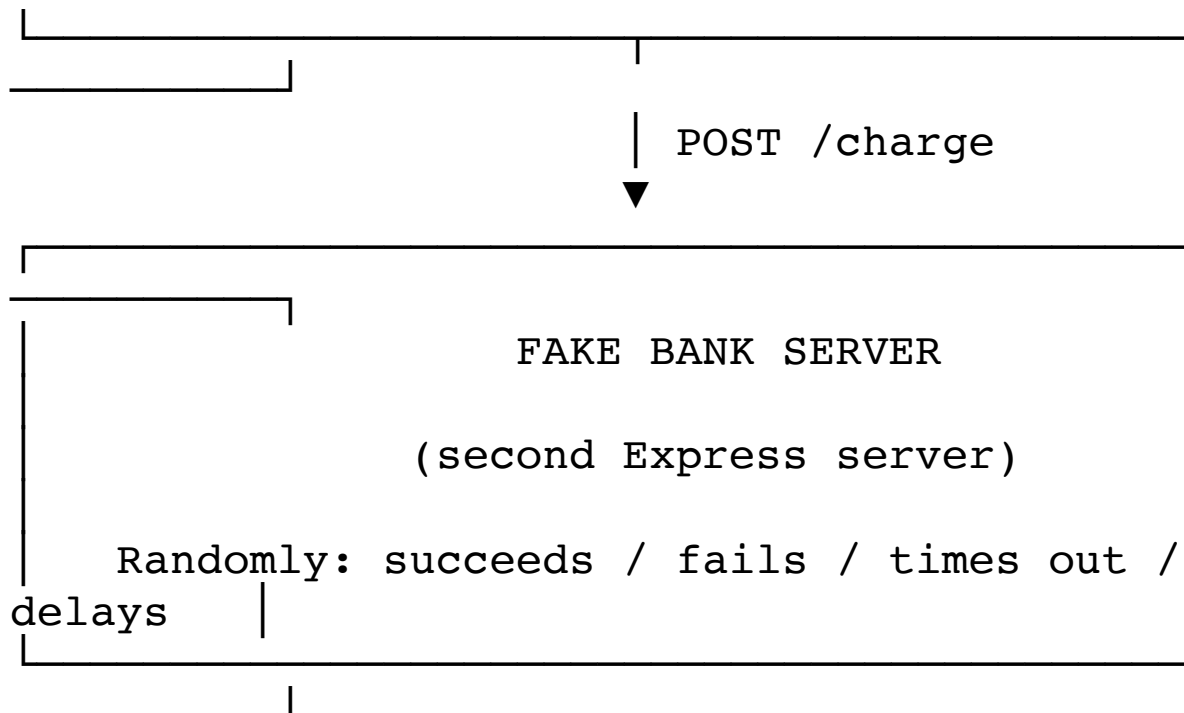
- Accepting payment requests from merchants
- Maintaining the exact state of every payment
- Communicating with a fake bank you also build
- Handling every failure mode correctly
- Retrying failed webhooks reliably
- Reconciling records at end of day

This is not a Stripe clone UI. There is no flashy frontend. The product is the system — the reliability, the correctness, the decisions.

End Product — What Exists When You Are Done







Everything runs locally with one command: **docker-compose up**

Stack

Runtime: Node.js
Framework: Express
Database: PostgreSQL
Cache/Queue: Redis + BullMQ
Container: Docker + Docker Compose
Reverse proxy: Nginx (Phase 5)
Language: JavaScript (you can switch to TypeScript in Phase 3)

The Six Phases

Phase 1 — Foundation (Core API + Database)

Phase 2 — Fake Bank + Payment Lifecycle

Phase 3 — Idempotency + Retry Logic

Phase 4 — Redis + Queues

Phase 5 — Docker + Nginx

Phase 6 — Reconciliation + Observability + Polish

PHASE 1 — Foundation

Goal: A running Express server connected to PostgreSQL. One working API endpoint. No bank yet. No Redis. Just the foundation.

What you build:

- Project folder structure
- PostgreSQL database with correct schema
- Express server with proper middleware
- POST `/payments` — creates a payment record, returns it
- GET `/payments/:id` — fetch payment by ID
- GET `/payments` — list all payments for a merchant (paginated)
- Merchant auth via API key middleware

Folder Structure

```
payment-gateway/  
├── src/  
│   ├── index.js           ← entry point,  
│   │   └── starts server  
│   └── app.js             ← Express app  
│       ├── setup, middleware  
│       └── config/  
│           └── db.js       ← PostgreSQL  
└── pool setup  
    └── middleware/
```

```

|   |   |— auth.js           ← merchant API
key verification
|   |   |— errorHandler.js  ← global error
handler
|   |   |— requestLogger.js  ← log every
request
|   |— routes/
|   |   |— payments.js       ← payment
routes
|   |— controllers/
|   |   |— paymentController.js
|   |— services/
|   |   |— paymentService.js ← business
logic, DB queries
|— db/
|   |— schema.sql           ← all CREATE
TABLE statements
|— .env.example
|— .env                     ← never commit
this
|— package.json
|— DECISIONS.md             ← write every
architectural decision here

```

Database Schema (Phase 1)

Run this in psql. Understand every line before running it.

```

-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Merchants who use your gateway
CREATE TABLE merchants (
    id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
    name        TEXT NOT NULL,

```

```

    email          TEXT UNIQUE NOT NULL,
    api_key        TEXT UNIQUE NOT NULL DEFAULT
encode(gen_random_bytes(32), 'hex'),
    webhook_url    TEXT,
    created_at     TIMESTAMP DEFAULT NOW()
);

```

-- Every payment attempt

```

CREATE TABLE payments (
    id              UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
    merchant_id     UUID NOT NULL REFERENCES
merchants(id),
    idempotency_key TEXT NOT NULL,
    amount          DECIMAL(12, 2) NOT NULL,
    currency        TEXT NOT NULL DEFAULT
'INR',
    status          TEXT NOT NULL DEFAULT
'INITIATED',
    customer_email  TEXT,
    description     TEXT,
    metadata        JSONB,
    bank_reference  TEXT,          -- bank's
transaction ID (filled later)
    failure_reason  TEXT,          -- why it
failed (filled later)
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW(),

```

-- Every payment for a merchant must have
unique idempotency key

```

    CONSTRAINT uq_merchant_idempotency UNIQUE
(merchant_id, idempotency_key),
    -- Status must be one of these exactly
    CONSTRAINT valid_status CHECK (

```

```
        status IN ('INITIATED', 'PROCESSING',
'SUCCESS', 'FAILED', 'PENDING')
    ),
    -- Amount must be positive
    CONSTRAINT positive_amount CHECK (amount >
0)
);
```

-- Audit log – every status change recorded here

```
CREATE TABLE payment_events (
    id          UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
    payment_id  UUID NOT NULL REFERENCES
payments(id),
    from_status TEXT,
    to_status   TEXT NOT NULL,
    reason      TEXT,
    metadata    JSONB,
    created_at  TIMESTAMP DEFAULT NOW()
);
```

-- Indexes for fast lookups

```
CREATE INDEX idx_payments_merchant      ON
payments(merchant_id, created_at DESC);
CREATE INDEX idx_payments_idempotency  ON
payments(merchant_id, idempotency_key);
CREATE INDEX idx_payments_status       ON
payments(status);
CREATE INDEX idx_events_payment        ON
payment_events(payment_id);
```

-- Seed one test merchant

```
INSERT INTO merchants (name, email,
webhook_url)
```

```
VALUES ('Test Merchant',  
       'merchant@test.com', 'http://localhost:4000/  
webhook')  
RETURNING id, api_key;  
-- Save the returned api_key — you'll use it  
in every request
```

Why each decision:

- • UUID not SERIAL — distributed systems, no sequential ID guessing
- • idempotency_key unique per merchant — same merchant can't double-charge
- • payment_events separate table — never lose history of what happened
- • CHECK constraints — database enforces business rules, not just application code
- • DECIMAL(12,2) not FLOAT — money must never have floating point errors

API Endpoints — Phase 1

Auth middleware — every request to `/payments` must include:

Header: `x-api-key: <merchant_api_key>`

Middleware looks up merchant by `api_key`. If not found → 401.

Attaches merchant to `req.merchant`. Routes never touch the key again.

POST /payments

Request:

```
{  
  "idempotency_key": "order_abc123",  
  "amount": 999.00,  
  "currency": "INR",  
  "customer_email": "customer@example.com",  
}
```

```
    "description": "Pro subscription",
    "metadata": { "order_id": "ORD-123" }
}
```

Logic:

- 1 1 Validate body — amount required and positive,
 idempotency_key required
- 2 2 Check if payment with this idempotency_key already
 exists for this merchant
- 3 3 If exists → return existing payment with X-
 Idempotent-Replay: true header
- 4 4 If not → create payment with status INITIATED
- 5 5 Insert into payment_events:
 from_status: null, to_status:
 'INITIATED'
- 6 6 Return 201 with payment object

Response:

```
{
  "id": "uuid",
  "status": "INITIATED",
  "amount": 999.00,
  "currency": "INR",
  "created_at": "2026-07-19T10:00:00Z"
}
```

GET /payments/:id

Logic:

- 1 1 Fetch payment by ID
- 2 2 Verify it belongs to the requesting merchant
 (important — merchants can't see each other's payments)
- 3 3 Return payment with its events array

GET /payments

Logic:

- | | | |
|---|---|---|
| 1 | 1 | Fetch all payments for this merchant |
| 2 | 2 | Support query params: ?
status=SUCCESS&limit=20&offset=0 |
| 3 | 3 | Return with total count for pagination |

What to Learn Today Alongside Building

When you hit these in Phase 1, go to Hussein Nasser backend course:

- • **Connection pooling** — why new `Pool()` not `new Client()` per request
- • **Parameterized queries** — why `$1`, `$2` not string concatenation (SQL injection)
- • **Transactions in pg** — `BEGIN`, `COMMIT`, `ROLLBACK` pattern
- • **DECIMAL vs FLOAT** — why money must use `DECIMAL`

Phase 1 Done When:

- ☐ `psql` opens, schema runs without errors
- ☐ Server starts on port 3000
- ☐ `POST /payments` with valid `api_key` → 201 with payment object
- ☐ `POST /payments` with same `idempotency_key` → same payment back
- ☐ `GET /payments/:id` → payment with events array
- ☐ `GET /payments` → list with pagination
- ☐ Any request without `api_key` → 401
- ☐ `payment_events` has one row per payment created
- ☐ `DECISIONS.md` has at least 5 entries explaining your choices

PHASE 2 — Fake Bank + Payment Lifecycle

Goal: Payments actually go somewhere. The bank responds. Your gateway handles success, failure, and timeout correctly.

What you build:

- Fake bank server (separate Express app on port 5000)
- POST `/payments/:id/process` — triggers payment processing
- State machine logic — INITIATED → PROCESSING → SUCCESS/FAILED/PENDING
- Webhook sending — notify merchant when payment reaches terminal state

Fake Bank Server

Separate folder: `fake-bank/`

```
// fake-bank/src/index.js
// This server simulates a real bank's
behavior
```

POST `/charge`

Body: `{ amount, currency, payment_id }`

Behavior (random):

- 65% → success after 200-800ms delay
- 20% → failure (`insufficient_funds / card_declined`)
- 15% → no response at all (simulates timeout)

Success response:

```
{ status: 'success', bank_reference:
'BANK-uuid' }
```

Failure response:

```
{ status: 'failed', reason:
'insufficient_funds' }
```

GET /transactions/:bank_reference

Returns transaction status – used for
PENDING resolution

Make the randomness configurable via environment variables so you
can force specific scenarios during testing:

BANK_BEHAVIOR=always_success

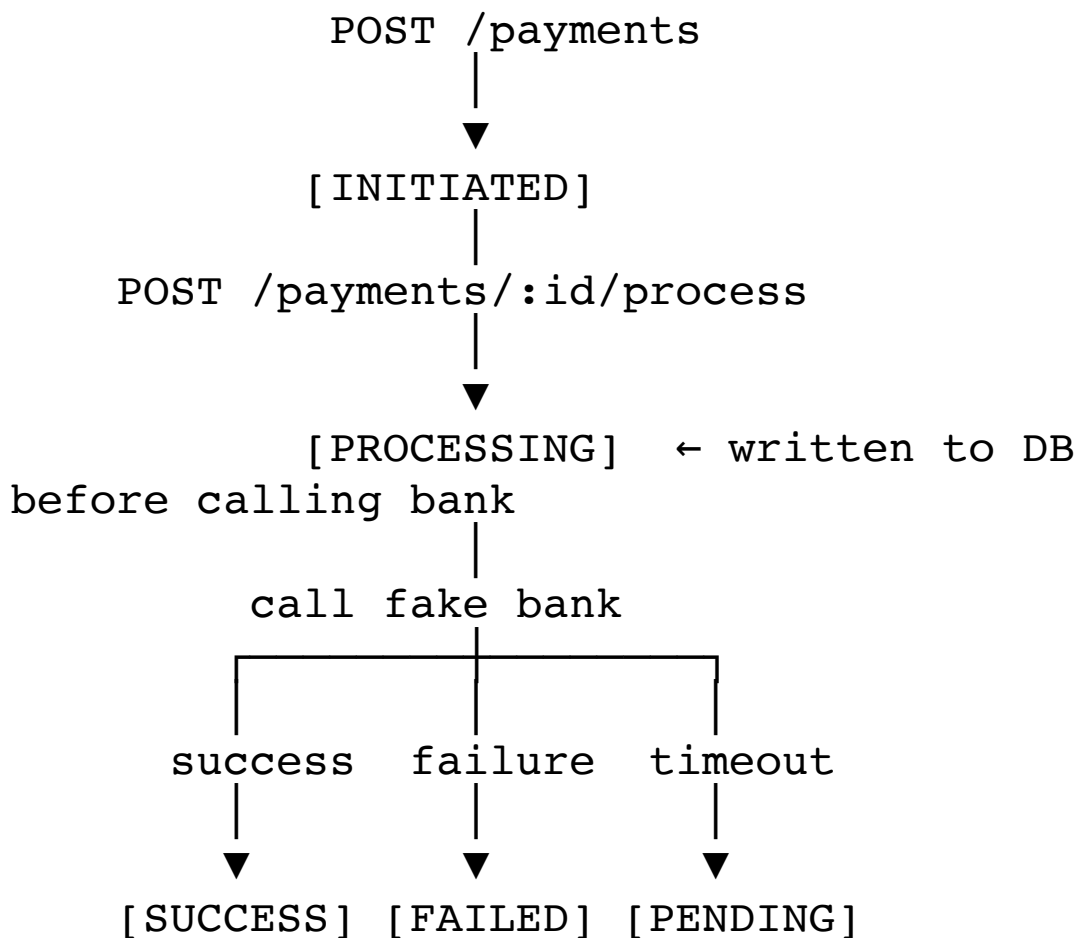
BANK_BEHAVIOR=always_fail

BANK_BEHAVIOR=always_timeout

BANK_BEHAVIOR=random (default)

Payment State Machine

This is the core of the gateway. Every payment follows this path:



```

      |
retry job checks
bank status later

```

```

[ SUCCESS ]    [ FAILED ]

```

Critical rule: Write PROCESSING to DB BEFORE calling the bank. Never call the bank while still in INITIATED. If your server crashes after the bank call but before DB write, you need to know the payment was in flight.

Status Transition Function

```

// src/services/paymentService.js

async function transitionStatus(client,
paymentId, fromStatus, toStatus, reason =
null, metadata = null) {
  // 1. Verify current status matches
  expected fromStatus
  const { rows } = await client.query(
    'SELECT status FROM payments WHERE id =
$1 FOR UPDATE',
    [paymentId]
  )

  if (rows[0].status !== fromStatus) {
    throw new Error(`Invalid transition:
expected ${fromStatus}, got $
${rows[0].status}`)
  }

  // 2. Update payment status
  await client.query(

```

```

        'UPDATE payments SET status = $1,
updated_at = NOW() WHERE id = $2',
        [toStatus, paymentId]
    )

    // 3. Record the event – never skip this
    await client.query(
        'INSERT INTO payment_events (payment_id,
from_status, to_status, reason, metadata)
VALUES ($1, $2, $3, $4, $5)',
        [paymentId, fromStatus, toStatus,
reason, metadata]
    )
}

```

Every status change goes through this function. No direct UPDATE to status anywhere else.

Processing Flow

```

// POST /payments/:id/process

async function processPayment(paymentId,
merchantId) {
    const client = await pool.connect()

    try {
        await client.query('BEGIN')

        // 1. Get payment, verify it belongs to
merchant, lock the row
        const payment = await
getPaymentForUpdate(client, paymentId,
merchantId)

        // 2. Can only process INITIATED

```

```

payments
  if (payment.status !== 'INITIATED') {
    throw new Error(`Cannot process
payment in status: ${payment.status}`)
  }

  // 3. Move to PROCESSING – written to DB
before bank call
    await transitionStatus(client,
paymentId, 'INITIATED', 'PROCESSING')

    await client.query('COMMIT')

    // 4. Call fake bank – OUTSIDE the
transaction
    // Why: transaction should be short.
Bank call can take seconds.
    const bankResult = await
callBankWithTimeout(payment, 10000) // 10s
timeout

    // 5. Handle bank response
const client2 = await pool.connect()
try {
  await client2.query('BEGIN')

  if (bankResult.status === 'success') {
    await client2.query(
      'UPDATE payments SET
bank_reference = $1 WHERE id = $2',
      [bankResult.bank_reference,
paymentId]
    )
    await transitionStatus(client2,
paymentId, 'PROCESSING', 'SUCCESS')
  }
}

```

```

        await sendWebhook(payment,
'SUCCESS')

    } else if (bankResult.status ===
'failed') {
        await client2.query(
            'UPDATE payments SET
failure_reason = $1 WHERE id = $2',
            [bankResult.reason, paymentId]
        )
        await transitionStatus(client2,
paymentId, 'PROCESSING', 'FAILED',
bankResult.reason)
        await sendWebhook(payment, 'FAILED')
    }

    await client2.query('COMMIT')
} finally {
    client2.release()
}

} catch (err) {
    await client.query('ROLLBACK')

    // If bank timed out – move to PENDING,
not FAILED
    if (err.code === 'BANK_TIMEOUT') {
        await moveToPending(paymentId, 'Bank
did not respond within timeout')
    } else {
        throw err
    }
} finally {
    client.release()
}

```

```
}
```

Webhook Sending (Phase 2 — Simple Version)

For now, just a direct HTTP call. In Phase 4, this becomes a queue.

```
async function sendWebhook(payment, status)
{
  if (!payment.merchant.webhook_url) return

  try {
    await
    axios.post(payment.merchant.webhook_url, {
      event: 'payment.updated',
      payment_id: payment.id,
      status: status,
      amount: payment.amount,
      timestamp: new Date().toISOString()
    }, { timeout: 5000 })
  } catch (err) {
    // Log failure but don't crash the
    payment flow
    // In Phase 4 this becomes a retry queue
    console.error('Webhook delivery
failed:', err.message)
  }
}
```

Phase 2 Done When:

- ☐ Fake bank runs on port 5000
- ☐ POST /payments/:id/process → payment moves through state machine
- ☐ Run 20 payments — see mix of SUCCESS, FAILED, PENDING in DB
- ☐ PENDING payments have a record in

payment_events

- No payment ever skips a status (INITIATED → SUCCESS is impossible)
- Forced timeout scenario → payment moves to PENDING, not crashed server
- Webhook fires on SUCCESS and FAILED (merchant test server receives it)

PHASE 3 — Idempotency + Retry Logic

Goal: The system handles repeated requests correctly. Failed webhooks retry automatically. PENDING payments get resolved.

What you build:

- Proper idempotency key storage (Redis-backed)
- Webhook retry queue (BullMQ)
- PENDING resolution cron job
- Switch to TypeScript (optional but recommended)

Idempotency (Properly)

Phase 1 used a DB unique constraint. That's good but not complete. A duplicate request that arrives while the first is still PROCESSING will pass the DB check (no completed result yet) and process twice.

Fix with Redis:

```
async function
acquireIdempotencyLock(merchantId,
idempotencyKey) {
  const key = `idempotency_lock:${
    merchantId}:${idempotencyKey}`

  // SET key value NX EX seconds
  // NX = only set if not exists
  // EX = expire after 30 seconds
```

```

    const result = await redis.set(key,
'locked', 'NX', 'EX', 30)

    return result === 'OK' // true = we got
the lock, false = already processing
}

```

```

async function
storeIdempotencyResult(merchantId,
idempotencyKey, paymentId) {
    const key = `idempotency_result:${
{merchantId}:${idempotencyKey}`}
    await redis.setex(key, 86400,
paymentId) // 24 hour TTL
}

```

```

async function
getIdempotencyResult(merchantId,
idempotencyKey) {
    const key = `idempotency_result:${
{merchantId}:${idempotencyKey}`}
    return redis.get(key)
}

```

Flow on POST /payments:

- | | | |
|---|---|--|
| 1 | 1 | Check Redis for existing result → return it if found |
| 2 | 2 | Try to acquire lock → if fails, return 409 (request in progress) |
| 3 | 3 | Process payment |
| 4 | 4 | Store result in Redis |
| 5 | 5 | Release lock |

Webhook Retry Queue

When webhook delivery fails in Phase 2, it's just logged. Now failed webhooks go into a BullMQ queue with retry logic.

```

// src/jobs/webhookWorker.js
const { Queue, Worker } = require('bullmq')

const webhookQueue = new Queue('webhooks', {
  connection: redis,
  defaultJobOptions: {
    attempts: 5,
    backoff: {
      type: 'exponential',
      delay: 2000 // 2s, 4s, 8s, 16s, 32s
    }
  }
})

// Add to queue instead of direct call
async function queueWebhook(payment, status)
{
  await webhookQueue.add('deliver', {
    payment_id: payment.id,
    merchant_id: payment.merchant_id,
    webhook_url:
payment.merchant.webhook_url,
    status,
    amount: payment.amount,
    timestamp: new Date().toISOString()
  })
}

// Worker that actually delivers
const worker = new Worker('webhooks', async
(job) => {
  const { webhook_url, ...payload } =
job.data

  const response = await

```

```

axios.post(webhook_url, payload, { timeout:
5000 })

    if (response.status < 200 ||
response.status >= 300) {
        throw new Error(`Webhook returned $
{response.status}`)
    }

    // Log successful delivery
    await
logWebhookDelivery(job.data.payment_id,
'delivered', job.attemptsMade)
}, { connection: redis })

worker.on('failed', async (job, err) => {
    if (job.attemptsMade >= job.opts.attempts)
    {
        // Moved to dead letter – alert needed
        await
logWebhookDelivery(job.data.payment_id,
'dead_letter', job.attemptsMade)
    }
})

```

PENDING Resolution Job

Runs every 2 minutes. Finds PENDING payments and checks with fake bank.

```

// src/jobs/pendingResolutionJob.js

async function resolvePendingPayments() {
    // Find payments stuck in PENDING for more
    than 2 minutes
    const { rows } = await pool.query(`

```

```

SELECT p.*, m.webhook_url
FROM payments p
JOIN merchants m ON p.merchant_id = m.id
WHERE p.status = 'PENDING'
AND p.updated_at < NOW() - INTERVAL '2
minutes'
LIMIT 50
`)

```

```

for (const payment of rows) {
  try {
    // Ask fake bank what happened
    const result = await
checkBankStatus(payment.bank_reference)

    if (result.status === 'success') {
      await resolvePayment(payment.id,
'SUCCESS')
      await queueWebhook(payment,
'SUCCESS')
    } else if (result.status === 'failed')
{
      await resolvePayment(payment.id,
'FAILED', result.reason)
      await queueWebhook(payment,
'FAILED')
    }
    // If still unknown – leave as
PENDING, job runs again in 2 mins

  } catch (err) {
    console.error(`Failed to resolve
payment ${payment.id}:`, err.message)
  }
}

```

```
}
```

```
// Schedule with setInterval or node-cron  
setInterval(resolvePendingPayments, 2 * 60 *  
1000)
```

Phase 3 Done When:

- Send same POST /payments twice simultaneously → only one payment created
- POST /payments while first is PROCESSING → 409, not a duplicate
- Webhook fails → appears in BullMQ queue → retries with backoff
- After 5 failures → appears in dead_letter queue
- PENDING payments get resolved within 4 minutes of creation
- Run 50 payments, check DB → every payment has a terminal status
- Zero payments stuck forever

PHASE 4 — Redis Integration

Goal: Add rate limiting, caching, metrics.

What you add:

- • Rate limiting per merchant (Redis sliding window)
- • Payment status caching (read-heavy optimization)
- • Basic metrics (success rate, volume, p95 latency)

Rate Limiting

```
// middleware/rateLimiter.js  
async function rateLimitMiddleware(req, res,  
next) {
```

```

    const merchantId = req.merchant.id
    const key = `rate_limit:${merchantId}:${
Math.floor(Date.now() / 60000)}`

    const count = await redis.incr(key)
    if (count === 1) await redis.expire(key,
60) // expire after 1 minute

    if (count > 100) { // 100 requests per
minute per merchant
        return res.status(429).json({
            error: 'Rate limit exceeded',
            retry_after: 60 - (Date.now() %
60000) / 1000
        })
    }

    res.setHeader('X-RateLimit-Limit', 100)
    res.setHeader('X-RateLimit-Remaining',
Math.max(0, 100 - count))
    next()
}

```

Payment Status Cache

```

async function getPaymentCached(paymentId,
merchantId) {
    const cacheKey = `payment:${paymentId}`

    const cached = await redis.get(cacheKey)
    if (cached) {
        const payment = JSON.parse(cached)
        // Verify merchant ownership even from
cache
        if (payment.merchant_id !== merchantId)

```

```

throw new ForbiddenError()
  return { ...payment, _cached: true }
}

const payment = await
db.getPayment(paymentId)

// Only cache terminal states – PENDING/
PROCESSING change frequently
if (['SUCCESS',
'FAILED'].includes(payment.status)) {
  await redis.setex(cacheKey, 300,
JSON.stringify(payment)) // 5 min TTL
}

return payment
}

```

Phase 4 Done When:

- 101 requests in one minute from one merchant → 429 on 101st
- GET payment after SUCCESS → served from cache on repeat calls
- PROCESSING payments never served from stale cache
- Basic metrics endpoint works: GET /metrics

PHASE 5 — Docker + Nginx

Goal: One command starts everything. Production-like setup.

What you build:

- Dockerfile for gateway
- Dockerfile for fake bank

- • docker-compose.yml wiring everything together
- • Nginx as reverse proxy

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  gateway:
```

```
    build: .
```

```
    ports:
```

```
      - "3000:3000"
```

```
    env_file: .env
```

```
    depends_on:
```

```
      postgres:
```

```
        condition: service_healthy
```

```
      redis:
```

```
        condition: service_healthy
```

```
    restart: unless-stopped
```

```
  fake-bank:
```

```
    build: ./fake-bank
```

```
    ports:
```

```
      - "5000:5000"
```

```
    restart: unless-stopped
```

```
  postgres:
```

```
    image: postgres:16-alpine
```

```
    environment:
```

```
      POSTGRES_DB: payments
```

```
      POSTGRES_USER: postgres
```

```
      POSTGRES_PASSWORD: postgres
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/
```

```
data
```

```
    - ./db/schema.sql:/docker-entrypoint-  
initdb.d/schema.sql  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U  
postgres"]  
      interval: 5s  
      retries: 5
```

```
redis:  
  image: redis:7-alpine  
  volumes:  
    - redis_data:/data  
  healthcheck:  
    test: ["CMD", "redis-cli", "ping"]  
    interval: 5s  
    retries: 5
```

```
nginx:  
  image: nginx:alpine  
  ports:  
    - "80:80"  
  volumes:  
    - ./nginx/nginx.conf:/etc/nginx/  
nginx.conf  
  depends_on:  
    - gateway
```

```
volumes:  
  postgres_data:  
  redis_data:
```

Phase 5 Done When:

- ☐ docker-compose up starts all 5 services
- ☐ All requests go through nginx on port 80

- ❑ docker-compose down then up → all data persists
- ❑ Gateway health check at GET /health → all services green
- ❑ README has "getting started in one command" section

PHASE 6 — Reconciliation + Observability + Polish

Goal: The system is complete and explainable.

What you build:

- • Daily reconciliation job
- • Structured JSON logging
- • Metrics endpoint
- • Load test
- • Final README

Reconciliation Job

Runs at midnight. Compares your DB with fake bank records.

```
async function runReconciliation() {
  const date = new
Date().toISOString().split('T')[0]

  // All payments your gateway recorded
today as SUCCESS
  const gatewayPayments = await pool.query(`
    SELECT id, bank_reference, amount
    FROM payments
    WHERE status = 'SUCCESS'
    AND DATE(created_at) = $1
  `, [date])
```

```

    // All transactions fake bank recorded
today
    const bankTransactions = await
getBankTransactionsForDate(date)

    const gatewayMap = new
Map(gatewayPayments.rows.map(p =>
[p.bank_reference, p]))
    const bankMap = new
Map(bankTransactions.map(t => [t.reference,
t]))

    const results = {
        matched: [],
        gateway_only: [],    // We think it
succeeded, bank has no record
        bank_only: [],      // Bank charged, we
have no record
        amount_mismatch: [] // Both have it but
amounts differ
    }

    for (const [ref, payment] of gatewayMap) {
        const bankTx = bankMap.get(ref)
        if (!bankTx) {
            results.gateway_only.push(payment)
        } else if (bankTx.amount !==
payment.amount) {

results.amount_mismatch.push({ payment,
bankTx })
        } else {
            results.matched.push(payment)
        }
    }

```

```

    }

    for (const [ref, tx] of bankMap) {
        if (!gatewayMap.has(ref))
results.bank_only.push(tx)
    }

    // Store report
    await pool.query(
        'INSERT INTO reconciliation_reports
(date, results, created_at) VALUES ($1, $2,
NOW())',
        [date, JSON.stringify(results)]
    )

    console.log(`Reconciliation ${date}: $
{results.matched.length} matched, $
{results.gateway_only.length}
discrepancies`)
}

```

Final README Structure

This is your interview document. Write it seriously.

Payment Gateway

What this is

One paragraph. What does it do and why is it interesting.

Architecture

The diagram from the top of this document.

Technical decisions

For each:

- Decision: Why UUID not SERIAL for payment IDs
- Reasoning: ...
- Tradeoff: ...

Do this for 8-10 decisions.

How to run
docker-compose up
That's it.

Problems I hit and solved
This section is gold in interviews.

- Problem: Double charges from concurrent requests
Solution: ...
- Problem: Lost payments when server crashed between bank call and DB write
Solution: ...

List 5-7 real problems you hit while building.

Load test results
ab -n 1000 -c 50 http://localhost/payments
Show the numbers.

What I would do at 10x scale
Show you understand the limits of what you built.

Phase 6 Done When:

- ☐ Reconciliation runs, produces a report, flags discrepancies
- ☐ Every request logs structured JSON with

payment_id, merchant_id, duration, status

- Load test: 1000 concurrent requests, zero data corruption
- README explains every major decision
- DECISIONS.md has 15+ entries
- You can explain any part of this in an interview without looking at notes

Today — Day 1

Do exactly this. Nothing more, nothing less.

1. Create the repo on GitHub

```
mkdir payment-gateway && cd payment-gateway && git init
```
2. Create the folder structure (empty files are fine)
3. Install dependencies

```
npm init -y
npm install express pg dotenv uuid
```
4. Create .env.example

```
DATABASE_URL=postgresql://
postgres:postgres@localhost:5432/payments
PORT=3000
```
5. Create the database in psql

```
createdb payments
```
6. Run the schema.sql

```
psql -d payments -f db/schema.sql
```
7. Verify schema

```
psql -d payments -c "\dt"
```

You should see: merchants, payments,
payment_events

8. Write src/config/db.js – the pg Pool
setup

Just this file. Make it connect and
export the pool.

9. Write a test query

```
const result = await pool.query('SELECT  
NOW()')
```

```
console.log(result.rows[0])
```

Run it. See a timestamp. Pool works.

10. Commit: "Phase 1: database schema and
pool setup"

That's Day 1. Schema designed, database running, pool connecting.

Tomorrow: Express server + first route.

DECISIONS.md Template

Create this file now. Fill it as you build.

Architectural Decisions

001 – UUID over SERIAL for payment IDs

Date:

Decision: Using UUID PRIMARY KEY not SERIAL

Reasoning:

Tradeoff:

002 – DECIMAL(12,2) for amounts

Date:

Decision:

Reasoning:

Tradeoff:

003 — Separate payment_events table

Date:

Decision:

Reasoning:

Tradeoff:

Phase 1 starts today. Everything else follows from here.